

```
/* ===== 文件: game.js ===== */
/**
 * 抖音小游戏入口文件
 * game.json 中约定入口为 game.js
 */
// ⚠️ 侧边栏复访官方指南要求: 必须在 game.js 运行时机「同步」注册 tt.onShow 监听,
// 注册过晚会收不到从侧边栏复访进入的启动回调, 导致用户无法领取奖励。
const platform = require('./js/platform.js');
platform.init();

const Main = require('./js/main.js');

const main = new Main();

// 浏览器预览/演示环境暴露实例 (抖音环境无 window, 自动跳过)
if (typeof window !== 'undefined') window.__game = main;
/* ===== 文件: js/config.js ===== */
/**
 * 全局配置常量
 * 引力方块: 20x20 方形棋盘, 方块从中心生成, 重力方向随机 (下/左/上/右)
 */

// 重力方向单位向量: 0=下 1=左 2=上 3=右
const DIRS = [
  { x: 0, y: 1, name: 'down', label: '↓' },
  { x: -1, y: 0, name: 'left', label: '←' },
  { x: 0, y: -1, name: 'up', label: '↑' },
  { x: 1, y: 0, name: 'right', label: '→' },
];

// 每个重力方向对应的“横向移动”轴 (与重力垂直)
// 重力向下/上时, 横向 = x 轴; 重力向左/右时, 横向 = y 轴
const PERP = [
  { x: 1, y: 0 }, // 重力向下: 横向为屏幕左右
  { x: 0, y: 1 }, // 重力向左: 横向为屏幕上下
  { x: 1, y: 0 }, // 重力向上: 横向为屏幕左右
  { x: 0, y: 1 }, // 重力向右: 横向为屏幕上下
];

// 七种方块的颜色
const COLORS = {
  I: '#00e5ff',
  O: '#ffd500',
  T: '#b388ff',
  S: '#69f0ae',
  Z: '#ff5252',
  J: '#448aff',
  L: '#ffab40',
};
```

```
/* ===== 关卡合格分 =====
* 每一关都有一个「合格分」（累计分数目标）：本局累计分数达到当前关合格分即算过关，
* 升入下一关（下落提速、提示缩短）。想调整某一关多少分合格，直接改下表对应项即可。
* 表格覆盖前 10 关；超出表格的关卡按公式外推：第 n 关需再净得 LEVEL_TARGET_STEP × n 分。
* 默认曲线：第 1 关 500，第 2 关 1500，第 3 关 3000，第 4 关 5000 ... (逐关增量 +500)
*/
const LEVEL_TARGETS = [500, 1500, 3000, 5000, 7500, 10500, 14000, 18000, 22500, 27500];
const LEVEL_TARGET_STEP = 500;

/** 第 level 关的合格分（累计分数目标）；level 从 1 开始 */
function levelTarget(level) {
  if (level <= LEVEL_TARGETS.length) return LEVEL_TARGETS[level - 1];
  let t = LEVEL_TARGETS[LEVEL_TARGETS.length - 1];
  for (let n = LEVEL_TARGETS.length + 1; n <= level; n++) t += LEVEL_TARGET_STEP * n;
  return t;
}

/* ===== 平台能力配置 =====
* 对应审核检测的五项能力：侧边栏复访 / 添加到桌面 / 订阅消息 / 广告 / 内购。
* 所有标 TODO 的 ID 必须替换为「抖音开放平台开发者后台」中创建的真实 ID，
* 否则对应能力在真机上会走失败回调（已做守卫降级，不影响游戏运行）。
*/
const PLATFORM = {
  /* ---- 广告：后台「流量主」→「广告位管理」创建后替换 ---- */
  REWARDED_AD_UNIT_ID: 'd9tbmu1k2kk5ehnpv1', // TODO: 替换为你的激励视频广告位 ID
  INTERSTITIAL_AD_UNIT_ID: 'e7fm9np91su7694n9q', // TODO: 替换为你的插屏广告位 ID
  INTERSTITIAL_MIN_INTERVAL: 90000, // 两次插屏的最小间隔 (ms)，防打扰

  /* ---- 订阅消息：后台「功能」→「订阅消息」选用模板后替换 ---- */
  SUBSCRIBE_TMPL_IDS: ['REPLACE_WITH_TMPL_ID'], // TODO: 替换为订阅消息模板 ID (可多个)

  /* ---- 内购：需游戏版号并在后台开通内购能力后才可开启 ---- */
  // 默认 false：内购调用代码已接入 (js/platform.js)，但购买入口隐藏，
  // 避免在无版号时触发「未获资质开通虚拟支付」的审核风险。拿到版号后改为 true。
  ENABLE_IAP: false,
  IAP: {
    productId: 'coins_100', // TODO: 后台配置的内购道具 ID
    priceFen: 100, // 道具价格 (单位：分)
    coins: 100, // 到账金币数
    zoneId: '1', // 游戏分区
  },

  /* ---- 金币经济（复活 / 奖励发放与消耗） ---- */
  WELCOME_COINS: 30, // 新玩家赠送金币（足够一次金币复活，保证复活链路可用）
  REVIVE_COIN_COST: 30, // 金币复活消耗
  AD_REWARD_COINS: 50, // 完整观看一次激励视频的金币奖励
  SIDEBAR_REWARD_COINS: 60, // 从抖音首页侧边栏进入游戏后可领取（每日一次）
};

module.exports = {
```

```
COLS: 20,
ROWS: 20,
TYPES: ['I', 'O', 'T', 'S', 'Z', 'J', 'L'],
DIRS,
PERP,
COLORS,

// 下落前提示 (方块类型 + 方向) 的停留时间
HINT_BASE: 1000,    // 第 1 关提示时长 ms
HINT_MIN: 500,      // 提示时长下限 ms
HINT_STEP: 50,      // 每升 1 关减少 ms

// 下落速度 (每格间隔) - 20x20 棋盘下落距离翻倍, 间隔减半以保持节奏与原 10x10 相当
FALL_BASE: 500,     // 第 1 关每格 ms
FALL_MIN: 80,       // 速度上限 (最快每格 80ms)
FALL_STEP: 40,      // 每升 1 关加快 ms

// 关卡合格分: 累计分数达到 LEVEL_TARGETS[level-1] 即过该关 (详见文件顶部说明)
LEVEL_TARGETS,
LEVEL_TARGET_STEP,
levelTarget,

SCORE_TABLE: [0, 100, 250, 500, 800], // 同时消除 1/2/3/4 行的基础分
SCORE_EXTRA: 200,                      // 超过 4 行后每多 1 行的附加基础分
SOFT_DROP_SCORE: 1,                    // 自然下落每格 +1
HARD_DROP_SCORE: 2,                    // 快速降落每格 +2

// 视觉
ACCENT: '#ff4d6d',
BG_TOP: '#0f1220',
BG_BOTTOM: '#1c2140',
BOARD_SCALE: 1.0,                      // 棋盘整体缩放 (方块大小): 20x20 下单格已约为 10x10 时的一半; 调小可进一步缩小方块

// 平台能力 (审核检测项) 配置
PLATFORM,
};
/* ===== 文件: js/tetromino.js ===== */
/**
 * 方块 (Tetromino) 定义与旋转, 纯逻辑, 可在 Node 中测试
 */

// 标准 7 种方块, 使用正方形矩阵便于旋转
const SHAPES = {
  I: [
    [0, 0, 0, 0],
    [1, 1, 1, 1],
    [0, 0, 0, 0],
    [0, 0, 0, 0],
  ],
  O: [
```

```
[1, 1],
[1, 1],
],
T: [
  [0, 1, 0],
  [1, 1, 1],
  [0, 0, 0],
],
S: [
  [0, 1, 1],
  [1, 1, 0],
  [0, 0, 0],
],
Z: [
  [1, 1, 0],
  [0, 1, 1],
  [0, 0, 0],
],
J: [
  [1, 0, 0],
  [1, 1, 1],
  [0, 0, 0],
],
L: [
  [0, 0, 1],
  [1, 1, 1],
  [0, 0, 0],
],
];

/** 顺时针旋转一个正方形矩阵 */
function rotateCW(m) {
  const n = m.length;
  const out = [];
  for (let r = 0; r < n; r++) {
    out.push([]);
    for (let c = 0; c < n; c++) {
      out[r][c] = m[n - 1 - c][r];
    }
  }
  return out;
}

/** 提取矩阵中所有实体格子的局部坐标 [{x:列, y:行}] */
function cellsOf(m) {
  const cells = [];
  for (let r = 0; r < m.length; r++) {
    for (let c = 0; c < m[r].length; c++) {
      if (m[r][c]) cells.push({ x: c, y: r });
    }
  }
}
```

```
    }
    return cells;
}

/** 复制矩阵 */
function cloneMatrix(m) {
    return m.map((row) => row.slice());
}

module.exports = { SHAPES, rotateCW, cellsOf, cloneMatrix };
/* ===== 文件: js/board.js ===== */
/**
 * 棋盘 (网格) 逻辑, 纯逻辑, 可在 Node 中测试
 * grid[y][x] 为 null (空) 或方块类型字母 (已固定)
 */

class Board {
    constructor(cols, rows) {
        this.cols = cols;
        this.rows = rows;
        this.grid = [];
        this.reset();
    }

    reset() {
        this.grid = [];
        for (let r = 0; r < this.rows; r++) {
            this.grid.push(new Array(this.cols).fill(null));
        }
    }

    inside(x, y) {
        return x >= 0 && x < this.cols && y >= 0 && y < this.rows;
    }

    occupied(x, y) {
        return !!this.grid[y][x];
    }

    /** 一组绝对坐标是否与边界或已固定格子冲突 */
    collides(cells) {
        for (let i = 0; i < cells.length; i++) {
            const c = cells[i];
            if (!this.inside(c.x, c.y)) return true;
            if (this.grid[c.y][c.x]) return true;
        }
        return false;
    }

    /** 把一组绝对坐标固定到棋盘上 */
}
```

```
lock(cells, type) {
  for (let i = 0; i < cells.length; i++) {
    const c = cells[i];
    if (this.inside(c.x, c.y)) this.grid[c.y][c.x] = type;
  }
}

fullRow(r) {
  for (let c = 0; c < this.cols; c++) {
    if (!this.grid[r][c]) return false;
  }
  return true;
}

fullCol(c) {
  for (let r = 0; r < this.rows; r++) {
    if (!this.grid[r][c]) return false;
  }
  return true;
}

/**
 * 消除所有被填满的行与列（四向重力下两个方向都可消除）
 * 返回 { rowCount, colCount, count, cells }, cells 为被消除格子（去重）
 */
clearLines() {
  const rows = [];
  const cols = [];
  for (let r = 0; r < this.rows; r++) if (this.fullRow(r)) rows.push(r);
  for (let c = 0; c < this.cols; c++) if (this.fullCol(c)) cols.push(c);

  const seen = {};
  const cells = [];
  const add = (x, y) => {
    const key = x + ',' + y;
    if (!seen[key]) {
      seen[key] = true;
      cells.push({ x, y });
    }
  };
  rows.forEach((r) => {
    for (let c = 0; c < this.cols; c++) add(c, r);
  });
  cols.forEach((c) => {
    for (let r = 0; r < this.rows; r++) add(c, r);
  });
  cells.forEach((p) => {
    this.grid[p.y][p.x] = null;
  });
}
```

```
    return { rowCount: rows.length, colCount: cols.length, count: rows.length + cols.length, cells };
}

/**
 * 重力沉降：每列的已固定格子整体向下落底，填满列内空洞（保持上下相对顺序）。
 * 用于消除后让剩余方块下沉，保证列内不再悬空、堆叠不会从中间裂开。
 * 返回 { moved, moves }：moved 为发生移动的格子数，
 * moves 为 [{x, fromY, toY, type}]（仅含实际移动的格子）。
 */
settle() {
    let moved = 0;
    const moves = [];
    for (let c = 0; c < this.cols; c++) {
        let write = this.rows - 1; // 下一个实体格应落的行（自底向上）
        for (let r = this.rows - 1; r >= 0; r--) {
            const v = this.grid[r][c];
            if (!v) continue;
            if (write !== r) {
                this.grid[write][c] = v;
                this.grid[r][c] = null;
                moved++;
                moves.push({ x: c, fromY: r, toY: write, type: v });
            }
            write--;
        }
    }
    return { moved, moves };
}

module.exports = Board;
/* ===== 文件: js/gamecore.js ===== */
/**
 * 游戏核心逻辑（纯 JS，无平台依赖，可在 Node 中单元测试）
 *
 * 相位机：idle → hint(下落前提示) → fall(下落中) → 锁定/消行+沉降 → 生成下一个
 * 方块从棋盘中心生成，重力方向随机（下/左/上/右），填满整行或整列即消除。
 * 消除后场上剩余方块整体下沉落底（每列向下压实），下沉凑齐的新行列连锁消除。
 */
const {
    COLS, ROWS, DIRS, PERP, TYPES,
    HINT_BASE, HINT_MIN, HINT_STEP,
    FALL_BASE, FALL_MIN, FALL_STEP,
    levelTarget: configLevelTarget,
    SCORE_TABLE, SCORE_EXTRA,
    SOFT_DROP_SCORE, HARD_DROP_SCORE,
} = require('./config.js');
const { SHAPES, rotateCW, cellsOf, cloneMatrix } = require('./tetromino.js');
const Board = require('./board.js');
```

```
// 旋转时依次尝试的踢墙偏移
const KICKS = [
  { x: 0, y: 0 }, { x: -1, y: 0 }, { x: 1, y: 0 },
  { x: 0, y: -1 }, { x: 0, y: 1 },
  { x: -2, y: 0 }, { x: 2, y: 0 }, { x: 0, y: -2 }, { x: 0, y: 2 },
];

class GameCore {
  constructor(opts) {
    opts = opts || {};
    this.rng = opts.rng || Math.random; // 可注入随机源, 便于测试
    this.events = []; // 事件队列: lock / clear / levelup / gameover
    this.board = new Board(COLS, ROWS);
    this.reset();
  }

  reset() {
    this.board.reset();
    this.score = 0;
    this.level = 1;
    this.lines = 0;
    this.gameOver = false;
    this.bag = [];
    this.next = this._drawNext();
    this.current = null;
    this.phase = 'idle';
    this.hintTimer = 0;
    this.fallTimer = 0;
    this.spawn();
  }

  /* ----- 随机出块: 7-bag (每 7 个一块袋, 袋内洗牌) ----- */

  _drawType() {
    if (this.bag.length === 0) {
      this.bag = TYPES.slice();
      for (let i = this.bag.length - 1; i > 0; i--) {
        const j = Math.floor(this.rng() * (i + 1));
        const t = this.bag[i];
        this.bag[i] = this.bag[j];
        this.bag[j] = t;
      }
    }
    return this.bag.pop();
  }

  _drawNext() {
    return { type: this._drawType(), dir: Math.floor(this.rng() * 4) };
  }
}
```

```
/* ----- 关卡参数 ----- */

/** 第 level 关的合格分（累积分目标）；缺省为当前关 */
levelTarget(level) {
  return configLevelTarget(level === undefined ? this.level : level);
}

hintTime() {
  return Math.max(HINT_MIN, HINT_BASE - (this.level - 1) * HINT_STEP);
}

fallInterval() {
  return Math.max(FALL_MIN, FALL_BASE - (this.level - 1) * FALL_STEP);
}

/**
 * 合格判定：累积分达到当前关合格分即过关 1 关（分数一次跨越多条合格线时连升）。
 * 在每次加分后调用；升级会推入 levelup 事件（level = 升入的新关卡）。
 */
_syncLevel() {
  while (this.score >= this.levelTarget(this.level)) {
    this.level++;
    this.events.push({ type: 'levelup', level: this.level, target: this.levelTarget(this.level) });
  }
}

/* ----- 坐标换算 ----- */

/** 某个方块占据的绝对格子坐标 */
cells(piece) {
  piece = piece || this.current;
  if (!piece) return [];
  const local = cellsOf(piece.matrix);
  const out = [];
  for (let i = 0; i < local.length; i++) {
    out.push({ x: local[i].x + piece.x, y: local[i].y + piece.y });
  }
  return out;
}

/** 当前方块沿重力方向的落点（幽灵方块） */
ghostCells() {
  if (!this.current) return [];
  const d = DIRS[this.current.dir];
  let gx = this.current.x;
  let gy = this.current.y;
  for (;;) {
    const cand = this._moved(this.current, d.x, d.y);
    cand.x = gx + d.x;
    cand.y = gy + d.y;
```

```
    if (this.board.collides(this.cells(cand))) break;
    gx = cand.x;
    gy = cand.y;
  }
  const local = cellsOf(this.current.matrix);
  const out = [];
  for (let i = 0; i < local.length; i++) out.push({ x: local[i].x + gx, y: local[i].y + gy });
  return out;
}

_moved(piece, dx, dy, matrix) {
  return {
    type: piece.type,
    matrix: matrix || piece.matrix,
    dir: piece.dir,
    x: piece.x + dx,
    y: piece.y + dy,
  };
}

/* ----- 生成与相位 ----- */

spawn() {
  const n = this.next;
  this.next = this._drawNext();
  const matrix = cloneMatrix(SHAPES[n.type]);
  const piece = {
    type: n.type,
    matrix,
    dir: n.dir,
    // 棋盘中心生成
    x: Math.floor((COLS - matrix.length) / 2),
    y: Math.floor((ROWS - matrix.length) / 2),
  };
  if (this.board.collides(this.cells(piece))) {
    // 中心被堵住 → 游戏结束
    this.current = piece;
    this.phase = 'over';
    this.gameOver = true;
    this.events.push({ type: 'gameover' });
    return;
  }
  this.current = piece;
  this.phase = 'hint';
  this.hintTimer = this.hintTime();
  this.fallTimer = 0;
}

canControl() {
  return !this.gameOver && !this.current && (this.phase === 'hint' || this.phase === 'fall');
```

```
}

/** 主循环驱动: dt 为毫秒 */
update(dt) {
  if (this.gameOver || !this.current) return;
  if (this.phase === 'hint') {
    this.hintTimer -= dt;
    if (this.hintTimer <= 0) this.phase = 'fall';
    return;
  }
  if (this.phase === 'fall') {
    this.fallTimer += dt;
    const iv = this.fallInterval();
    let guard = 0;
    while (this.fallTimer >= iv && guard++ < 64) {
      this.fallTimer -= iv;
      if (!this._stepGravity(true)) break; // 落底锁定
    }
  }
}

/** 沿重力方向前进一格; 无法前进则锁定。返回是否移动成功 */
_stepGravity(withSoftScore) {
  const d = DIRS[this.current.dir];
  const moved = this._moved(this.current, d.x, d.y);
  if (this.board.collides(this.cells(moved))) {
    this._lock();
    return false;
  }
  this.current = moved;
  if (withSoftScore) {
    this.score += SOFT_DROP_SCORE;
    this._syncLevel(); // 软降分同样计入合格进度
  }
  return true;
}

/* ----- 玩家操作 ----- */

/** 沿与重力垂直的轴移动, sign = -1 / +1 */
movePerp(sign) {
  if (!this.canControl()) return false;
  const p = PERP[this.current.dir];
  const moved = this._moved(this.current, p.x * sign, p.y * sign);
  if (this.board.collides(this.cells(moved))) return false;
  this.current = moved;
  return true;
}

/** 顺时针旋转 (带踢墙) */
```

```
rotate() {
  if (!this.canControl()) return false;
  if (this.current.type === '0') return true; // 0 旋转无意义
  const m = rotateCW(this.current.matrix);
  for (let i = 0; i < KICKS.length; i++) {
    const cand = this._moved(this.current, KICKS[i].x, KICKS[i].y, m);
    if (!this.board.collides(this.cells(cand))) {
      this.current = cand;
      return true;
    }
  }
  return false;
}

/** 快速降落: 沿重力方向直接落底锁定 */
hardDrop() {
  if (!this.canControl()) return;
  this.phase = 'fall';
  let dist = 0;
  while (this._stepGravity(false)) dist++;
  this.score += dist * HARD_DROP_SCORE;
  this._syncLevel();
}

/* ----- 锁定 / 消行 / 升级 ----- */

_lock() {
  const cells = this.cells(this.current);
  this.board.lock(cells, this.current.type);
  this.events.push({ type: 'lock', cells: cells.slice() });

  const cleared = this.board.clearLines();
  if (cleared.count > 0) {
    // 消除 → 沉降 循环: 消除后剩余方块整体下沉 (每列向下压实, 不再悬空);
    // 下沉可能凑齐新的整行/整列 → 继续消除并沉降, 直到稳定 (连锁消除合并计数)
    let count = cleared.count;
    const allCells = cleared.cells.slice();
    let settledMoved = 0;
    for (;;) {
      const settled = this.board.settle();
      settledMoved += settled.moved;
      const next = this.board.clearLines();
      if (next.count === 0) break;
      count += next.count;
      for (let i = 0; i < next.cells.length; i++) allCells.push(next.cells[i]);
    }

    let base = SCORE_TABLE[Math.min(count, 4)];
    if (count > 4) base += (count - 4) * SCORE_EXTRA;
    this.score += base * this.level;
  }
}
```

```
this.lines += count;
this.events.push({ type: 'clear', cells: allCells, count, settled: settledMoved });

    this._syncLevel(); // 累计分数达到当前关合格分 → 过关升级
}

this.current = null;
this.phase = 'idle';
this.spawn(); // 生成下一个 (可能触发 gameover)
}

/**
 * 复活: 清空棋盘并重新出块 (保留分数/关卡/行数) 。
 * 仅在游戏结束后可调用一次; 返回是否复活成功。
 * 供「看激励视频复活 / 金币复活」使用 (见 main.js tryRevive) 。
 */
revive() {
    if (!this.gameOver) return false;
    this.board.reset();
    this.events.length = 0; // 丢弃残留事件 (含 gameover)
    this.gameOver = false;
    this.bag = [];
    this.next = this._drawNext();
    this.current = null;
    this.phase = 'idle';
    this.hintTimer = 0;
    this.fallTimer = 0;
    this.spawn();
    return !this.gameOver;
}

/** 取走并清空事件队列 */
drainEvents() {
    const ev = this.events;
    this.events = [];
    return ev;
}
}

module.exports = GameCore;
/* ===== 文件: js/render.js ===== */
/**
 * 渲染模块: 全部 Canvas 2D 绘制逻辑
 * 场景: menu / playing / paused / gameover / rank / help
 */
const {
    COLS, ROWS, DIRS, PERP, COLORS, ACCENT, BG_TOP, BG_BOTTOM, BOARD_SCALE, PLATFORM,
} = require('./config.js');
const { SHAPES, cellsOf } = require('./tetromino.js');
```

```

/* ===== 布局 ===== */

function buildLayout(w, h) {
    const s = Math.max(0.66, Math.min(1.25, Math.min(w / 375, h / 700)));

    const hud = { x: 12 * s, y: 10 * s, w: w - 24 * s, h: 52 * s };
    const pauseBtn = { x: w - 54 * s, y: 16 * s, w: 40 * s, h: 40 * s };
    const nextBox = { x: 12 * s, y: 68 * s, w: 128 * s, h: 64 * s };
    const gravBox = { x: 148 * s, y: 68 * s, w: w - 160 * s, h: 64 * s };

    // 棋盘: 正方形, 尽量大
    const reserved = 322 * s; // HUD + 提示行 + 控制区
    let bs = Math.min(w - 24 * s, h - reserved, h * 0.56);
    bs = Math.max(bs, Math.min(180, w - 24 * s));
    bs *= BOARD_SCALE; // 整体缩放棋盘: 方块更小, 四周留白更多
    let by = 142 * s;
    const slack = h - 96 * s - bs - by;
    if (slack > 0) by += slack * 0.3;
    const board = { x: (w - bs) / 2, y: by, size: bs, cell: bs / COLS };

    // 底部控制按钮
    const btnSize = Math.min(64 * s, (w - 40 * s) / 4.6);
    const cy = h - 92 * s;
    const xs = [w * 0.14, w * 0.38, w * 0.62, w * 0.86];
    const controls = {
        left: { x: xs[0] - btnSize / 2, y: cy, w: btnSize, h: btnSize },
        rotate: { x: xs[1] - btnSize / 2, y: cy, w: btnSize, h: btnSize },
        drop: { x: xs[2] - btnSize / 2, y: cy, w: btnSize, h: btnSize },
        right: { x: xs[3] - btnSize / 2, y: cy, w: btnSize, h: btnSize },
    };

    // 排行榜页好友区域
    const rankArea = { x: 16 * s, y: 104 * s, w: w - 32 * s, h: Math.min(280 * s, h * 0.38) };

    return { w, h, s, hud, pauseBtn, nextBox, gravBox, board, controls, rankArea };
}

/* ===== 按钮定义 (绘制 + 命中检测共用) ===== */

function mk(id, r, extra) {
    const b = { id: id, x: r.x, y: r.y, w: r.w, h: r.h };
    if (extra) for (const k in extra) b[k] = extra[k];
    return b;
}

/**
 * 场景按钮定义 (绘制 + 命中检测共用)
 * @param opts {canRevive, sidebarSupported, sidebarClaimable} 由 main.sceneOpts() 提供
 */
function sceneButtons(scene, L, opts) {

```

```

opts = opts || {};
const s = L.s, w = L.w, h = L.h;
const out = [];
if (scene === 'menu') {
  const bw = 220 * s, bh = 56 * s, bx = (w - bw) / 2;
  let y = h * 0.50;
  out.push(mk('start', { x: bx, y: y, w: bw, h: bh }, { text: '开始游戏', primary: true }));
  y += bh + 12 * s;
  out.push(mk('rank', { x: bx, y: y, w: bw, h: 48 * s }, { text: '排行榜' }));
  y += 48 * s + 12 * s;
  out.push(mk('help', { x: bx, y: y, w: bw, h: 48 * s }, { text: '玩法说明' }));
  y += 48 * s + 12 * s;
  // 侧边栏复访奖励入口 (必接能力; 宿主明确不支持时隐藏)
  if (opts.sidebarSupported !== false) {
    out.push(mk('sidebarGift', { x: bx, y: y, w: bw, h: 44 * s }, {
      text: opts.sidebarClaimable ? '侧边栏奖励 · 可领取' : '侧边栏奖励',
      primary: !opts.sidebarClaimable,
    }));
    y += 44 * s + 12 * s;
  }
  // 平台能力小按钮行: 添加到桌面 / 订阅提醒 / 免费金币 (看激励视频)
  const gap = 6 * s;
  const sw = (bw - gap * 2) / 3;
  const sh = 36 * s;
  out.push(mk('desktop', { x: bx, y: y, w: sw, h: sh }, { text: '添加到桌面', small2: true }));
  out.push(mk('subscribe', { x: bx + sw + gap, y: y, w: sw, h: sh }, { text: '订阅提醒', small2: true }));
  out.push(mk('freeCoins', { x: bx + (sw + gap) * 2, y: y, w: sw, h: sh }, { text: '免费金币', small2: true }));
} else if (scene === 'playing' || scene === 'paused') {
  const c = L.controls;
  out.push(mk('left', c.left, { glyph: 'perp', sign: -1 }));
  out.push(mk('rotate', c.rotate, { text: '旋转', small: true }));
  out.push(mk('drop', c.drop, { glyph: 'drop' }));
  out.push(mk('right', c.right, { glyph: 'perp', sign: 1 }));
  out.push(mk('pause', L.pauseBtn, { glyph: 'pause' }));
  if (scene === 'paused') {
    const bw = 200 * s, bh = 50 * s, bx = (w - bw) / 2, py = h * 0.42;
    out.push(mk('resume', { x: bx, y: py, w: bw, h: bh }, { text: '继续游戏', primary: true }));
    out.push(mk('restart', { x: bx, y: py + (bh + 14 * s), w: bw, h: bh }, { text: '重新开始' }));
    out.push(mk('tomenu', { x: bx, y: py + (bh + 14 * s) * 2, w: bw, h: bh }, { text: '返回主页' }));
  }
} else if (scene === 'gameover') {
  const bw = 220 * s, bh = 50 * s, bx = (w - bw) / 2;
  let y = opts.canRevive ? h * 0.42 : h * 0.47;
  if (opts.canRevive) {
    // 复活按钮 (看激励视频 / 金币复活, 见 main.tryRevive), 每局限一次
    out.push(mk('revive', { x: bx, y: y, w: bw, h: bh }, { text: '复活继续', primary: true }));
    y += bh + 14 * s;
    out.push(mk('retry', { x: bx, y: y, w: bw, h: bh }, { text: '再来一局' }));
  } else {
    out.push(mk('retry', { x: bx, y: y, w: bw, h: bh }, { text: '再来一局', primary: true }));
  }
}

```

```

    }
    y += bh + 14 * s;
    out.push(mk('rank', { x: bx, y: y, w: bw, h: bh }, { text: '排行榜' }));
    y += bh + 14 * s;
    out.push(mk('share', { x: bx, y: y, w: bw, h: bh }, { text: '分享给好友' }));
    y += bh + 14 * s;
    out.push(mk('tomenu', { x: bx, y: y, w: bw, h: bh }, { text: '返回主页' }));
  } else if (scene === 'rank' || scene === 'help') {
    const bw = 160 * s, bh = 46 * s;
    out.push(mk('back', { x: (w - bw) / 2, y: h - 78 * s, w: bw, h: bh }, { text: '返回' }));
  } else if (scene === 'sidebar') {
    // 侧边栏复访任务面板: 去侧边栏 / 领取奖励
    const bw = 220 * s, bh = 50 * s, bx = (w - bw) / 2;
    out.push(mk('sidebarAction', { x: bx, y: h * 0.565, w: bw, h: bh }, {
      text: opts.sidebarClaimable ? '立即领奖' : '去首页侧边栏',
      primary: true,
    }));
    const cw = 160 * s;
    out.push(mk('sidebarClose', { x: (w - cw) / 2, y: h * 0.645, w: cw, h: 44 * s }, { text: '关闭' }));
  }
  return out;
}

/* ===== 绘制小工具 ===== */

function roundRect(ctx, x, y, w, h, r) {
  if (w < 2 || h < 2) return;
  r = Math.min(r, w / 2, h / 2);
  ctx.beginPath();
  ctx.moveTo(x + r, y);
  ctx.arcTo(x + w, y, x + w, y + h, r);
  ctx.arcTo(x + w, y + h, x, y + h, r);
  ctx.arcTo(x, y + h, x, y, r);
  ctx.arcTo(x, y, x + w, y, r);
  ctx.closePath();
}

function drawTriangle(ctx, cx, cy, dx, dy, size, color) {
  const nx = -dy, ny = dx;
  ctx.beginPath();
  ctx.moveTo(cx + dx * size * 0.6, cy + dy * size * 0.6);
  ctx.lineTo(cx - dx * size * 0.4 + nx * size * 0.5, cy - dy * size * 0.4 + ny * size * 0.5);
  ctx.lineTo(cx - dx * size * 0.4 - nx * size * 0.5, cy - dy * size * 0.4 - ny * size * 0.5);
  ctx.closePath();
  ctx.fillStyle = color;
  ctx.fill();
}

function text(ctx, str, x, y, size, color, align, bold, alpha) {
  ctx.save();

```

```
ctx.font = (bold ? 'bold ' : '') + size + 'px sans-serif';
ctx.fillStyle = color || '#ffffff';
ctx.textAlign = align || 'left';
ctx.textBaseline = 'alphabetic';
if (alpha !== undefined) ctx.globalAlpha = alpha;
ctx.fillText(str, x, y);
ctx.restore();
}

/** 以 (cx,cy) 为中心绘制方块小图（按实体格包围盒居中） */
function drawMiniShape(ctx, type, cx, cy, cell, alpha) {
  const m = SHAPES[type];
  const cs = cellsOf(m);
  let minX = 99, maxX = -99, minY = 99, maxY = -99;
  for (let i = 0; i < cs.length; i++) {
    if (cs[i].x < minX) minX = cs[i].x;
    if (cs[i].x > maxX) maxX = cs[i].x;
    if (cs[i].y < minY) minY = cs[i].y;
    if (cs[i].y > maxY) maxY = cs[i].y;
  }
  const bw = (maxX - minX + 1) * cell;
  const bh = (maxY - minY + 1) * cell;
  const ox = cx - bw / 2 - minX * cell;
  const oy = cy - bh / 2 - minY * cell;
  ctx.save();
  ctx.globalAlpha = alpha === undefined ? 1 : alpha;
  for (let i = 0; i < cs.length; i++) {
    const px = ox + cs[i].x * cell;
    const py = oy + cs[i].y * cell;
    roundRect(ctx, px + 1, py + 1, cell - 2, cell - 2, cell * 0.22);
    ctx.fillStyle = COLORS[type];
    ctx.fill();
  }
  ctx.restore();
}

function drawCell(ctx, px, py, size, color, alpha) {
  ctx.save();
  ctx.globalAlpha = alpha === undefined ? 1 : alpha;
  roundRect(ctx, px + 1.5, py + 1.5, size - 3, size - 3, Math.min(7, size * 0.2));
  ctx.fillStyle = color;
  ctx.fill();
  // 顶部高光
  ctx.fillStyle = 'rgba(255,255,255,0.22)';
  roundRect(ctx, px + 4, py + 3.5, Math.max(2, size - 8), Math.max(2, size * 0.12), 2);
  ctx.fill();
  ctx.restore();
}

function drawButton(ctx, b, L, dir, enabled) {
```

```

const s = L.s;
ctx.save();
if (enabled === false) ctx.globalAlpha = 0.45;
if (b.primary) {
  const g = ctx.createLinearGradient(b.x, b.y, b.x, b.y + b.h);
  g.addColorStop(0, '#ff6b8a');
  g.addColorStop(1, '#e63958');
  roundRect(ctx, b.x, b.y, b.w, b.h, b.h * 0.28);
  ctx.fillStyle = g;
  ctx.fill();
} else {
  roundRect(ctx, b.x, b.y, b.w, b.h, Math.min(b.h * 0.28, 14 * s));
  ctx.fillStyle = 'rgba(255,255,255,0.09)';
  ctx.fill();
  ctx.strokeStyle = 'rgba(255,255,255,0.18)';
  ctx.lineWidth = 1;
  ctx.stroke();
}

const cx = b.x + b.w / 2;
const cy = b.y + b.h / 2;

if (b.glyph === 'perp' && dir !== undefined) {
  const p = PERP[dir];
  drawTriangle(ctx, cx, cy, p.x * b.sign, p.y * b.sign, Math.min(b.w, b.h) * 0.42, 'ffffff');
} else if (b.glyph === 'drop' && dir !== undefined) {
  const d = DIRS[dir];
  drawTriangle(ctx, cx, cy - 4 * s, d.x, d.y, Math.min(b.w, b.h) * 0.44, ACCENT);
  text(ctx, '落下', cx, cy + b.h * 0.34, 10 * s, 'rgba(255,255,255,0.75)', 'center', false);
} else if (b.glyph === 'pause') {
  const bw2 = 4 * s, bh2 = 14 * s, gap = 3 * s;
  ctx.fillStyle = 'ffffff';
  ctx.fillRect(cx - gap - bw2, cy - bh2 / 2, bw2, bh2);
  ctx.fillRect(cx + gap, cy - bh2 / 2, bw2, bh2);
} else if (b.text) {
  const fs = b.small2 ? 11 : (b.small ? 15 : 17);
  const dy = b.small2 ? 4 * s : (b.small ? 5 * s : 7 * s);
  text(ctx, b.text, cx, cy + dy, fs * s,
    b.primary ? 'ffffff' : 'rgba(255,255,255,0.92)', 'center', true);
}
ctx.restore();
}

/* ===== 场景绘制 ===== */

function drawBackground(ctx, L) {
  const g = ctx.createLinearGradient(0, 0, 0, L.h);
  g.addColorStop(0, BG_TOP);
  g.addColorStop(1, BG_BOTTOM);
  ctx.fillStyle = g;

```

```
    ctx.fillRect(0, 0, L.w, L.h);
}

function drawMenu(ctx, L, main, t) {
    const s = L.s, w = L.w, h = L.h;
    // 漂浮装饰方块
    const deco = [
        { type: 'T', x: w * 0.18, y: h * 0.16, cell: 13 * s, ph: 0 },
        { type: 'I', x: w * 0.80, y: h * 0.22, cell: 11 * s, ph: 2 },
        { type: 'S', x: w * 0.14, y: h * 0.36, cell: 10 * s, ph: 4 },
        { type: 'L', x: w * 0.86, y: h * 0.40, cell: 11 * s, ph: 1 },
    ];
    for (let i = 0; i < deco.length; i++) {
        const d = deco[i];
        const yy = d.y + Math.sin(t / 900 + d.ph) * 6 * s;
        drawMiniShape(ctx, d.type, d.x, yy, d.cell, 0.5);
    }

    text(ctx, '引力方块', w / 2, h * 0.30, 42 * s, 'ffffff', 'center', true);
    text(ctx, '四向重力 · 方块消除玩法', w / 2, h * 0.30 + 30 * s, 14 * s, 'rgba(255,255,255,0.65)', 'center', false);
    text(ctx, '最高分 ' + main.best + ' · 金币 ' + (main.coins || 0), w / 2, h * 0.50 - 22 * s,
        15 * s, 'rgba(255,215,0,0.9)', 'center', true);

    const btns = sceneButtons('menu', L, main.sceneOpts ? main.sceneOpts() : {});
    for (let i = 0; i < btns.length; i++) drawButton(ctx, btns[i], L, undefined, true);

    text(ctx, 'v1.2.0 · 抖音小游戏', w / 2, h - 20 * s, 11 * s, 'rgba(255,255,255,0.35)', 'center', false);
}

function drawHud(ctx, L, main) {
    const s = L.s, hud = L.hud;
    const core = main.core;
    const target = core.levelTarget(); // 当前关合格分 (累计分数目标)
    const colW = (hud.w - 56 * s) / 4;
    const cols = [
        { label: '分数', value: String(core.score), size: 22 },
        { label: '合格分', value: String(target), size: 16, color: '#ffd54f' },
        { label: '关卡', value: String(core.level), size: 20 },
        { label: '行数', value: String(core.lines), size: 20 },
    ];
    for (let i = 0; i < cols.length; i++) {
        const c = cols[i];
        const cx = hud.x + colW * i + colW / 2;
        const fs = c.value.length > 6 ? c.size - 4 : c.size; // 长数字自动缩小
        text(ctx, c.label, cx, hud.y + 16 * s, 11 * s, 'rgba(255,255,255,0.55)', 'center', false);
        text(ctx, c.value, cx, hud.y + 42 * s, fs * s, c.color || 'ffffff', 'center', true);
    }
    // 本关进度条: 上一关合格分 → 当前关合格分
    const prev = core.level > 1 ? core.levelTarget(core.level - 1) : 0;
    const k = Math.max(0, Math.min(1, (core.score - prev) / Math.max(1, target - prev)));
```

```
const bw = hud.w, bh = Math.max(3, 4 * s), bx = hud.x, by = hud.y + 50 * s;
roundRect(ctx, bx, by, bw, bh, bh / 2);
ctx.fillStyle = 'rgba(255,255,255,0.12)';
ctx.fill();
if (k > 0) {
  roundRect(ctx, bx, by, Math.max(bh, bw * k), bh, bh / 2);
  ctx.fillStyle = '#ffd54f';
  ctx.fill();
}
}

function drawNextBox(ctx, L, main, t) {
  const s = L.s, bx = L.nextBox;
  const next = main.core.next;
  roundRect(ctx, bx.x, bx.y, bx.w, bx.h, 10 * s);
  ctx.fillStyle = 'rgba(255,255,255,0.06)';
  ctx.fill();
  text(ctx, '下一个', bx.x + 10 * s, bx.y + 15 * s, 10 * s, 'rgba(255,255,255,0.55)', 'left', false);
  drawMiniShape(ctx, next.type, bx.x + bx.w * 0.38, bx.y + bx.h * 0.58, 10 * s, 1);
  const d = DIRS[next.dir];
  drawTriangle(ctx, bx.x + bx.w * 0.78, bx.y + bx.h * 0.58, d.x, d.y, 16 * s, ACCENT);
}

function drawGravBox(ctx, L, main, t) {
  const s = L.s, gx = L.gravBox;
  const cur = main.core.current;
  const dir = cur ? cur.dir : 0;
  const d = DIRS[dir];
  roundRect(ctx, gx.x, gx.y, gx.w, gx.h, 10 * s);
  ctx.fillStyle = 'rgba(255,255,255,0.06)';
  ctx.fill();
  text(ctx, '重力方向', gx.x + 12 * s, gx.y + 15 * s, 10 * s, 'rgba(255,255,255,0.55)', 'left', false);
  drawTriangle(ctx, gx.x + 30 * s, gx.y + gx.h * 0.60, d.x, d.y, 20 * s, ACCENT);
  const hinting = main.core.phase === 'hint';
  const stateText = main.core.gameOver ? '-' : (hinting ? '准备下落...' : '下落中');
  const alpha = hinting ? 0.55 + 0.4 * Math.sin(t / 160) : 0.75;
  text(ctx, stateText, gx.x + 56 * s, gx.y + gx.h * 0.66, 13 * s, 'rgba(255,255,255,' + alpha.toFixed(2) + ')', 'left', true);
}

function drawBoard(ctx, L, main, t) {
  const s = L.s, B = L.board, core = main.core;
  const cell = B.cell;

  // 底板
  roundRect(ctx, B.x - 4 * s, B.y - 4 * s, B.size + 8 * s, B.size + 8 * s, 10 * s);
  ctx.fillStyle = 'rgba(0,0,0,0.35)';
  ctx.fill();
  ctx.strokeStyle = 'rgba(255,255,255,0.10)';
  ctx.lineWidth = 1;
  ctx.stroke();
}
```

```
// 网格线
ctx.save();
ctx.strokeStyle = 'rgba(255,255,255,0.05)';
ctx.lineWidth = 1;
ctx.beginPath();
for (let i = 1; i < COLS; i++) {
  ctx.moveTo(B.x + i * cell, B.y);
  ctx.lineTo(B.x + i * cell, B.y + B.size);
  ctx.moveTo(B.x, B.y + i * cell);
  ctx.lineTo(B.x + B.size, B.y + i * cell);
}
ctx.stroke();
ctx.restore();

// 中心生成点标记
ctx.save();
ctx.strokeStyle = 'rgba(255,255,255,0.16)';
ctx.lineWidth = 1.5;
const ccx = B.x + B.size / 2, ccy = B.y + B.size / 2, cr = 5 * s;
ctx.beginPath();
ctx.moveTo(ccx - cr, ccy); ctx.lineTo(ccx + cr, ccy);
ctx.moveTo(ccx, ccy - cr); ctx.lineTo(ccx, ccy + cr);
ctx.stroke();
ctx.restore();

const px = (c) => B.x + c.x * cell;
const py = (c) => B.y + c.y * cell;

// 已固定格子
for (let r = 0; r < ROWS; r++) {
  for (let c = 0; c < COLS; c++) {
    const v = core.board.grid[r][c];
    if (v) drawCell(ctx, B.x + c * cell, B.y + r * cell, cell, COLORS[v], 1);
  }
}

const cur = core.current;
if (cur && !core.gameOver) {
  const dir = cur.dir;
  const d = DIRS[dir];

  // 重力侧墙高亮
  ctx.save();
  ctx.strokeStyle = ACCENT;
  ctx.globalAlpha = 0.85;
  ctx.lineWidth = 3 * s;
  ctx.beginPath();
  if (d.y === 1) { ctx.moveTo(B.x, B.y + B.size - 1.5 * s); ctx.lineTo(B.x + B.size, B.y + B.size - 1.5 * s); }
  if (d.y === -1) { ctx.moveTo(B.x, B.y + 1.5 * s); ctx.lineTo(B.x + B.size, B.y + 1.5 * s); }
```

```
if (d.x === 1) { ctx.moveTo(B.x + B.size - 1.5 * s, B.y); ctx.lineTo(B.x + B.size - 1.5 * s, B.y + B.size); }
if (d.x === -1) { ctx.moveTo(B.x + 1.5 * s, B.y); ctx.lineTo(B.x + 1.5 * s, B.y + B.size); }
ctx.stroke();
ctx.restore();

// 幽灵落点
const ghost = core.ghostCells();
ctx.save();
ctx.globalAlpha = 0.3;
ctx.strokeStyle = COLORS[cur.type];
ctx.lineWidth = 1.5;
for (let i = 0; i < ghost.length; i++) {
  roundRect(ctx, px(ghost[i]) + 2, py(ghost[i]) + 2, cell - 4, cell - 4, cell * 0.18);
  ctx.stroke();
}
ctx.restore();

// 当前方块 (hint 相位半透明呼吸闪烁 = 下落前提示)
const hinting = core.phase === 'hint';
const alpha = hinting ? 0.45 + 0.3 * (0.5 + 0.5 * Math.sin(t / 150)) : 1;
const cells = core.cells(cur);
for (let i = 0; i < cells.length; i++) {
  drawCell(ctx, px(cells[i]), py(cells[i]), cell, COLORS[cur.type], alpha);
}

// hint 相位: 大号方向箭头 (从中心指向重力墙)
if (hinting) {
  const pulse = 0.55 + 0.45 * Math.sin(t / 150);
  const ax = B.x + B.size / 2 + d.x * B.size * 0.33;
  const ay = B.y + B.size / 2 + d.y * B.size * 0.33;
  ctx.save();
  ctx.globalAlpha = pulse;
  drawTriangle(ctx, ax, ay, d.x, d.y, 34 * s, '#ffffff');
  ctx.restore();
}
}

// 消行闪光特效
for (let i = 0; i < main.fx.length; i++) {
  const f = main.fx[i];
  const k = 1 - f.t / f.dur;
  ctx.save();
  ctx.globalAlpha = Math.max(0, k) * 0.85;
  ctx.fillStyle = '#ffffff';
  for (let j = 0; j < f.cells.length; j++) {
    ctx.fillRect(px(f.cells[j]), py(f.cells[j]), cell, cell);
  }
  ctx.restore();
}
```

```
// 提示浮字
if (main.toast) {
  const k = main.toast.t / main.toast.dur;
  const alpha = k < 0.15 ? k / 0.15 : (k > 0.7 ? (1 - k) / 0.3 : 1);
  text(ctx, main.toast.text, B.x + B.size / 2, B.y - 12 * s - (1 - alpha) * 8 * s,
    17 * s, '#ffd54f', 'center', true, Math.max(0, alpha));
}
}

function drawControls(ctx, L, main) {
  const dir = main.core.current ? main.core.current.dir : 0;
  const enabled = main.core.canControl();
  const btns = sceneButtons('playing', L);
  for (let i = 0; i < btns.length; i++) {
    const b = btns[i];
    if (b.id === 'pause') { drawButton(ctx, b, L, dir, true); continue; }
    drawButton(ctx, b, L, dir, enabled);
  }
}

function drawGameScene(ctx, L, main, t) {
  drawHud(ctx, L, main);
  drawNextBox(ctx, L, main, t);
  drawGravBox(ctx, L, main, t);
  drawBoard(ctx, L, main, t);
  drawControls(ctx, L, main);
}

function drawOverlay(ctx, L, alpha) {
  ctx.save();
  ctx.fillStyle = 'rgba(5,7,15,' + alpha + ')';
  ctx.fillRect(0, 0, L.w, L.h);
  ctx.restore();
}

function drawPaused(ctx, L, main) {
  drawOverlay(ctx, L, 0.55);
  const s = L.s;
  text(ctx, '已暂停', L.w / 2, L.h * 0.32, 30 * s, '#ffffff', 'center', true);
  const btns = sceneButtons('paused', L);
  const dir = main.core.current ? main.core.current.dir : 0;
  for (let i = 0; i < btns.length; i++) {
    const b = btns[i];
    if (b.id === 'resume' || b.id === 'restart' || b.id === 'tomenu') drawButton(ctx, b, L, dir, true);
  }
}

function drawGameOver(ctx, L, main) {
  drawOverlay(ctx, L, 0.6);
  const s = L.s, w = L.w, h = L.h, core = main.core;
```

```

const px2 = 26 * s, pw = w - 52 * s;
const py2 = h * 0.14, ph = h * 0.30;
roundRect(ctx, px2, py2, pw, ph, 16 * s);
ctx.fillStyle = 'rgba(20,24,44,0.95)';
ctx.fill();
ctx.strokeStyle = 'rgba(255,255,255,0.12)';
ctx.lineWidth = 1;
ctx.stroke();

text(ctx, '游戏结束', w / 2, py2 + 40 * s, 24 * s, '#ffffff', 'center', true);
text(ctx, main.newBest ? '新纪录!' : '中心被堵住了', w / 2, py2 + 64 * s, 12 * s,
  main.newBest ? '#ffd54f' : 'rgba(255,255,255,0.55)', 'center', true);

const rows = [
  ['本局分数', String(core.score)],
  ['最高分', String(main.best)],
  ['到达关卡', '第 ' + core.level + ' 关'],
  ['消除行数', core.lines + ' 行'],
];
const ry0 = py2 + 92 * s;
for (let i = 0; i < rows.length; i++) {
  const yy = ry0 + i * 24 * s;
  text(ctx, rows[i][0], px2 + 28 * s, yy, 13 * s, 'rgba(255,255,255,0.6)', 'left', false);
  text(ctx, rows[i][1], px2 + pw - 28 * s, yy, 14 * s, '#ffffff', 'right', true);
}

const btns = sceneButtons('gameover', L, main.sceneOpts ? main.sceneOpts() : {});
for (let i = 0; i < btns.length; i++) drawButton(ctx, btns[i], L, undefined, true);
}

/** 侧边栏复访任务面板（必接能力，官方指引：入口奖励 + 跳转侧边栏 + 复访领奖） */
function drawSidebar(ctx, L, main) {
  drawOverlay(ctx, L, 0.66);
  const s = L.s, w = L.w, h = L.h;
  const px2 = 22 * s, pw = w - 44 * s;
  const py2 = h * 0.14, ph = h * 0.58;
  roundRect(ctx, px2, py2, pw, ph, 16 * s);
  ctx.fillStyle = 'rgba(20,24,44,0.97)';
  ctx.fill();
  ctx.strokeStyle = 'rgba(255,255,255,0.12)';
  ctx.lineWidth = 1;
  ctx.stroke();

  text(ctx, '首页侧边栏入口奖励', w / 2, py2 + 40 * s, 20 * s, '#ffffff', 'center', true);
  text(ctx, '每日从侧边栏进入游戏可领 ' + PLATFORM.SIDEBAR_REWARD_COINS + ' 金币',
    w / 2, py2 + 66 * s, 13 * s, 'rgba(255,215,0,0.9)', 'center', true);

  const lines = [
    '④ 点击下方「去首页侧边栏」按钮',
  ]

```

```

    '② 在侧边栏中点击「引力方块」图标',
    '③ 返回游戏, 点击「立即领奖」',
];
for (let i = 0; i < lines.length; i++) {
    text(ctx, lines[i], px2 + 26 * s, py2 + 106 * s + i * 30 * s, 14 * s, 'rgba(255,255,255,0.85)', 'left', false);
}

const opts = main.sceneOpts ? main.sceneOpts() : {};
let status, color;
if (opts.sidebarClaimable) {
    status = '✓ 已检测到侧边栏复访, 可领取今日奖励';
    color = '#69f0ae';
} else if (opts.sidebarClaimedToday) {
    status = '今日奖励已领取, 明天再来';
    color = 'rgba(255,255,255,0.5)';
} else if (opts.sidebarSupported === false) {
    status = '当前宿主不支持侧边栏, 请在抖音 App 中打开';
    color = 'rgba(255,255,255,0.5)';
} else {
    status = '尚未从侧边栏进入, 按上述步骤完成即可领奖';
    color = 'rgba(255,255,255,0.65)';
}
text(ctx, status, w / 2, h * 0.565 - 20 * s, 12 * s, color, 'center', true);

const btns = sceneButtons('sidebar', L, opts);
for (let i = 0; i < btns.length; i++) {
    const b = btns[i];
    const enabled = (b.id !== 'sidebarAction') || opts.sidebarSupported !== false;
    drawButton(ctx, b, L, undefined, enabled);
}
}

/** 全局浮动提示 (平台能力反馈等, 所有场景可见) */
function drawPToast(ctx, L, main) {
    const p = main.ptoast;
    if (!p) return;
    const k = p.t / p.dur;
    let alpha = 1;
    if (k < 0.1) alpha = k / 0.1;
    else if (k > 0.75) alpha = (1 - k) / 0.25;
    alpha = Math.max(0, Math.min(1, alpha));
    const s = L.s;
    ctx.save();
    ctx.globalAlpha = alpha;
    ctx.font = 'bold ' + (13 * s) + 'px sans-serif';
    const tw = ctx.measureText(p.text).width;
    const bw = tw + 36 * s, bh = 36 * s;
    const bx = (L.w - bw) / 2, by = L.h * 0.15;
    roundRect(ctx, bx, by, bw, bh, bh / 2);
    ctx.fillStyle = 'rgba(10,12,24,0.9)';

```

```
ctx.fill();
ctx.strokeStyle = 'rgba(255,255,255,0.18)';
ctx.lineWidth = 1;
ctx.stroke();
text(ctx, p.text, L.w / 2, by + bh * 0.68, 13 * s, 'ffffff', 'center', true);
ctx.restore();
}

function drawRank(ctx, L, main, t) {
  const s = L.s, w = L.w, h = L.h;
  text(ctx, '排行榜', w / 2, 44 * s, 24 * s, 'ffffff', 'center', true);

  // 好友排行 (开放数据域共享画布)
  const A = L.rankArea;
  roundRect(ctx, A.x, A.y, A.w, A.h, 12 * s);
  ctx.save();
  ctx.clip();
  ctx.fillStyle = 'rgba(255,255,255,0.05)';
  ctx.fillRect(A.x, A.y, A.w, A.h);
  if (main.friendCanvas && main.friendCanvas.width > 0 && main.friendCanvas.height > 0) {
    try {
      ctx.drawImage(main.friendCanvas, A.x, A.y, A.w, A.h);
    } catch (e) {
      text(ctx, '好友排行渲染失败', A.x + A.w / 2, A.y + A.h / 2, 13 * s, 'rgba(255,255,255,0.5)', 'center', false);
    }
  } else {
    text(ctx, '好友排行', A.x + A.w / 2, A.y + A.h / 2 - 10 * s, 14 * s, 'rgba(255,255,255,0.6)', 'center', true);
    text(ctx, '需登录抖音并在真机运行', A.x + A.w / 2, A.y + A.h / 2 + 12 * s, 11 * s, 'rgba(255,255,255,0.4)', 'center', false);
  }
  ctx.restore();

  // 本机 TOP10
  const ly = A.y + A.h + 26 * s;
  text(ctx, '本机 TOP10', A.x + 4 * s, ly, 14 * s, 'fffd54f', 'left', true);
  const list = main.localRank || [];
  if (list.length === 0) {
    text(ctx, '暂无记录, 快去玩一局吧', w / 2, ly + 34 * s, 12 * s, 'rgba(255,255,255,0.45)', 'center', false);
  } else {
    const n = Math.min(list.length, 8);
    const rowH = Math.min(24 * s, (h - 100 * s - ly) / Math.max(n, 1));
    for (let i = 0; i < n; i++) {
      const it = list[i];
      const yy = ly + 22 * s + i * rowH;
      const medal = i === 0 ? 'fffd54f' : (i === 1 ? 'cfd8dc' : (i === 2 ? 'ffab40' : 'rgba(255,255,255,0.45)'));
      text(ctx, String(i + 1), A.x + 10 * s, yy, 12 * s, medal, 'left', true);
      text(ctx, String(it.score), A.x + 40 * s, yy, 13 * s, 'ffffff', 'left', true);
      const dt = new Date(it.ts || Date.now());
      const dateStr = (dt.getMonth() + 1) + '-' + dt.getDate();
      text(ctx, '第' + it.level + '关 · ' + it.lines + '行 · ' + dateStr,
        A.x + A.w - 8 * s, yy, 11 * s, 'rgba(255,255,255,0.5)', 'right', false);
    }
  }
}
```

```
    }
  }

  const btns = sceneButtons('rank', L);
  for (let i = 0; i < btns.length; i++) drawButton(ctx, btns[i], L, undefined, true);
}

function drawHelp(ctx, L, main) {
  drawOverlay(ctx, L, 0.66);
  const s = L.s, w = L.w, h = L.h;
  const px2 = 22 * s, pw = w - 44 * s;
  const py2 = h * 0.10, ph = h * 0.66;
  roundRect(ctx, px2, py2, pw, ph, 16 * s);
  ctx.fillStyle = 'rgba(20,24,44,0.97)';
  ctx.fill();

  text(ctx, '玩法说明', w / 2, py2 + 38 * s, 20 * s, 'ffffff', 'center', true);
  const lines = [
    '· 方块从棋盘中心生成，箭头指示重力方向',
    '· 下落前方块会停留片刻，提示类型与方向',
    '· ◀ ▶ 按钮：沿垂直于重力的方向移动',
    '· 点击棋盘任意处：旋转方块',
    '· 沿重力方向滑动棋盘：快速落底',
    '· 「落下」按钮：立即沿重力落底',
    '· 填满任意整行或整列即可消除得分',
    '· 消除后剩余方块整体下沉落底，可连锁消除',
    '· 每关设有合格分：第 1 关 500 分、第 2 关 1500 分...',
    '· 累计分数达到合格分即过关，下落速度逐关加快',
    '· 中心出生点被堵住时游戏结束',
  ];
  for (let i = 0; i < lines.length; i++) {
    text(ctx, lines[i], px2 + 22 * s, py2 + 76 * s + i * 27 * s, 13 * s, 'rgba(255,255,255,0.85)', 'left', false);
  }

  const btns = sceneButtons('help', L);
  for (let i = 0; i < btns.length; i++) drawButton(ctx, btns[i], L, undefined, true);
}

/* ===== 总入口 ===== */

function draw(main) {
  const ctx = main.ctx, L = main.L, t = Date.now();
  drawBackground(ctx, L);
  switch (main.state) {
    case 'menu':
      drawMenu(ctx, L, main, t);
      break;
    case 'playing':
      drawGameScene(ctx, L, main, t);
      break;
  }
}
```

```
    case 'paused':
        drawGameScene(ctx, L, main, t);
        drawPaused(ctx, L, main);
        break;
    case 'gameover':
        drawGameScene(ctx, L, main, t);
        drawGameOver(ctx, L, main);
        break;
    case 'rank':
        drawRank(ctx, L, main, t);
        break;
    case 'help':
        drawMenu(ctx, L, main, t);
        drawHelp(ctx, L, main);
        break;
    case 'sidebar':
        drawMenu(ctx, L, main, t);
        drawSidebar(ctx, L, main);
        break;
    default:
        break;
}
drawPToast(ctx, L, main);
}

module.exports = { buildLayout, sceneButtons, draw, roundRect };
/* ===== 文件: js/input.js ===== */
/**
 * 触摸输入：区分“点按”与“滑动”手势
 * - 点按（位移小、时间短）→ onTap(x, y)
 * - 滑动（位移足够大）→ onSwipe(dx, dy, startX, startY)
 */
class Input {
    constructor(TT) {
        this.onTap = null;
        this.onSwipe = null;
        this._start = null;
        if (!TT) return;
        try {
            TT.onTouchStart((e) => this._onStart(e));
            TT.onTouchMove((e) => this._onMove(e));
            TT.onTouchEnd((e) => this._onEnd(e));
            TT.onTouchCancel(() => { this._start = null; });
        } catch (e) { /* 平台不支持时忽略 */ }
    }

    _touch(e) {
        return e && e.touches && e.touches[0] ? e.touches[0] : null;
    }
}
```

```
_onStart(e) {
  const t = this._touch(e);
  if (!t) return;
  this._start = { x: t.clientX, y: t.clientY, t: Date.now(), moved: false };
}

_onMove(e) {
  if (!this._start) return;
  const t = this._touch(e);
  if (!t) return;
  if (Math.abs(t.clientX - this._start.x) > 12 || Math.abs(t.clientY - this._start.y) > 12) {
    this._start.moved = true;
  }
}

_onEnd(e) {
  const s = this._start;
  this._start = null;
  if (!s) return;
  const t = e && e.changedTouches && e.changedTouches[0] ? e.changedTouches[0] : null;
  const x = t ? t.clientX : s.x;
  const y = t ? t.clientY : s.y;
  const dx = x - s.x;
  const dy = y - s.y;
  const dist = Math.sqrt(dx * dx + dy * dy);
  if (!s.moved && dist < 14 && Date.now() - s.t < 600) {
    if (this.onTap) this.onTap(x, y);
  } else if (dist >= 26) {
    if (this.onSwipe) this.onSwipe(dx, dy, s.x, s.y);
  }
}
}

module.exports = Input;
/* ===== 文件: js/platform.js ===== */
/**
 * 平台能力模块：统一封装抖音小游戏审核要求的平台能力
 *
 * 对应审核检测项：
 * 1. 侧边栏复访（必接） - tt.navigateToScene({scene:'sidebar'}) 跳转侧边栏；
 *   tt.checkScene 判断宿主是否支持；tt.onShow 监听启动来源（location='sidebar_card'
 *   或 scene='021036' 即为侧边栏启动），据此发放每日复访奖励。
 *   ⚠️ 官方要求 tt.onShow 必须在 game.js 运行时同步注册（见 game.js）。
 * 2. 添加到桌面 - tt.addToDesktop
 * 3. 订阅消息 - tt.requestSubscribeMessage（模板 ID 见 config.js PLATFORM）
 * 4. 广告 - tt.createRewardedVideoAd（激励视频） / tt.createInterstitialAd（插屏）
 * 5. 内购 - tt.requestMidasPaymentGameItem（需版号；PLATFORM.ENABLE_IAP=false 时入口隐藏）
 *
 * 另含金币钱包（tt.setStorageSync 持久化）：看广告得金币、侧边栏复访得金币、
 * 金币复活消耗、（开通内购后）购买金币。
 */
```

```
*
* 说明：本文件刻意使用字面量 tt.xxx(...) 形式调用（而非别名），
* 与官方文档/审核检测的调用特征保持一致；所有调用均先经 typeof 守卫 + try/catch，
* API 缺失或失败时静默降级，绝不影响游戏本体运行。
*/
const { PLATFORM } = require('./config.js');

const KEY_COINS = 'gravcube.coins';
const KEY_SIDEBAR_CLAIM = 'gravcube.sidebarClaim'; // 值为 'YYYY-M-D', 标记当日已领取
const KEY_WELCOMED = 'gravcube.welcomed';          // 新手赠币只发一次

/* ===== 能力检测 ===== */

const hasTT = (typeof tt !== 'undefined') && !!tt;

const api = {
  rewardedAd: hasTT && typeof tt.createRewardedVideoAd === 'function',
  interstitialAd: hasTT && typeof tt.createInterstitialAd === 'function',
  subscribe: hasTT && typeof tt.requestSubscribeMessage === 'function',
  addToDesktop: hasTT && typeof tt.addToDesktop === 'function',
  sidebar: hasTT && typeof tt.navigateToScene === 'function', // 审核要求：必须使用 tt.navigateToScene 调用
  checkScene: hasTT && typeof tt.checkScene === 'function',
  iap: hasTT && typeof tt.requestMidasPaymentGameItem === 'function',
  showModal: hasTT && typeof tt.showModal === 'function',
};

/* ===== 存储 ===== */

function safeGet(key, def) {
  try {
    if (hasTT && tt.getStorageSync) {
      const v = tt.getStorageSync(key);
      return (v === '' || v === undefined || v === null) ? def : v;
    }
  } catch (e) { /* 忽略存储异常 */ }
  return def;
}

function safeSet(key, value) {
  try {
    if (hasTT && tt.setStorageSync) tt.setStorageSync(key, value);
  } catch (e) { /* 忽略存储异常 */ }
}

/* ===== 金币钱包 ===== */

let coinsCache = null; // 内存缓存，避免每帧读存储

function getCoins() {
  if (coinsCache === null) {
```

```
    const v = safeGet(KEY_COINS, 0);
    coinsCache = (typeof v === 'number' && isFinite(v) && v > 0) ? Math.floor(v) : 0;
  }
  return coinsCache;
}

function addCoins(n) {
  const v = getCoins() + Math.max(0, Math.floor(n) || 0);
  coinsCache = v;
  safeSet(KEY_COINS, v);
  return v;
}

function spendCoins(n) {
  n = Math.floor(n) || 0;
  if (getCoins() < n) return false;
  coinsCache -= n;
  safeSet(KEY_COINS, coinsCache);
  return true;
}

/* ===== 启动信息与生命周期（侧边栏复访） ===== */

let launchInfo = null;          // 最近一次 tt.onShow 的启动参数（官方要求始终用最新值判断）
let sidebarAvailable = null;    // tt.checkScene 结果: true / false / null(未知)
const showListeners = [];
let lifecycleReady = false;

function todayStr() {
  const d = new Date();
  return d.getFullYear() + '-' + (d.getMonth() + 1) + '-' + d.getDate();
}

/** 是否从抖音首页侧边栏启动（官方判定: location='sidebar_card', scene='021036'） */
function isFromSidebar() {
  if (!launchInfo) return false;
  return launchInfo.location === 'sidebar_card' || launchInfo.scene === '021036';
}

/**
 * 初始化: 必须在 game.js 运行时尽早调用（官方文档强调 tt.onShow 注册过晚会收不到回调）
 */
function init() {
  if (lifecycleReady || !hasTT) return;
  lifecycleReady = true;

  // 冷启动参数（部分宿主 onShow 首次不回调，用同步接口兜底）
  try {
    if (tt.getLaunchOptionsSync) launchInfo = tt.getLaunchOptionsSync() || null;
  } catch (e) { /* 忽略 */ }
```

```
// 热启动/复访监听：始终保留最新启动信息
try {
  if (tt.onShow) {
    tt.onShow((opts) => {
      if (opts) launchInfo = opts;
      for (let i = 0; i < showListeners.length; i++) {
        try { showListeners[i](launchInfo); } catch (e) { /* 忽略监听器异常 */ }
      }
    });
  }
} catch (e) { /* 忽略 */ }

// 当前宿主是否支持跳转侧边栏 (isExist=false 时不展示奖励入口)
try {
  if (api.checkScene) {
    tt.checkScene({
      scene: 'sidebar',
      success: (res) => { sidebarAvailable = !(res && res.isExist); },
      fail: () => { sidebarAvailable = false; },
    });
  }
} catch (e) { /* 忽略 */ }

// 新手赠币（一次性）：保证「金币复活」链路在无广告环境下也可用
if (!safeGet(KEY_WELCOMED, false)) {
  safeSet(KEY_WELCOMED, true);
  if (PLATFORM.WELCOME_COINS > 0) addCoins(PLATFORM.WELCOME_COINS);
}
}

/** 注册 onShow 监听（如刷新金币显示） */
function onShow(cb) {
  if (typeof cb === 'function') showListeners.push(cb);
}

/* ===== 侧边栏复访 ===== */

let sidebarClaimCache = null; // 内存缓存最近领取日期，避免每帧读存储

function claimedDate() {
  if (sidebarClaimCache === null) sidebarClaimCache = safeGet(KEY_SIDEBAR_CLAIM, '');
  return sidebarClaimCache;
}

/**
 * 侧边栏复访状态（纯内存判断，可每帧调用）
 * supported    - 当前宿主是否支持（checkScene 明确返回 false 才算不支持）
 * fromSidebar  - 本次启动是否来自侧边栏
 * claimedToday - 今日是否已领取
 */
```

```
* claimable    - 当前是否可领取奖励
*/
function getSidebarState() {
    const supported = api.sidebar && sidebarAvailable !== false;
    const claimedToday = claimedDate() === todayStr();
    const fromSidebar = isFromSidebar();
    return {
        supported,
        fromSidebar,
        claimedToday,
        claimable: supported && fromSidebar && !claimedToday,
    };
}

/** 跳转抖音首页侧边栏 (审核要求: 必须使用 tt.navigateToScene 调用) */
function goSidebar(onResult) {
    const cb = onResult || function () {};
    if (!api.sidebar) { cb({ ok: false, reason: 'unsupported' }); return; }
    try {
        tt.navigateToScene({
            scene: 'sidebar', // 官方接入指引: scene 固定传 'sidebar'
            success: (res) => cb({ ok: true, res }),
            fail: (res) => cb({ ok: false, reason: 'fail', res }),
        });
    } catch (e) {
        cb({ ok: false, reason: 'error', err: e });
    }
}

/** 领取侧边栏复访奖励 (每日一次), 成功返回 true */
function claimSidebarReward() {
    const st = getSidebarState();
    if (!st.claimable) return false;
    sidebarClaimCache = todayStr();
    safeSet(KEY_SIDEBAR_CLAIM, sidebarClaimCache);
    addCoins(PLATFORM.SIDEBAR_REWARD_COINS);
    return true;
}

/* ===== 激励视频广告 ===== */

let rewardedAd = null;
let rewardedPending = null; // 当前等待结果的回调 (防并发)

function failRewardedPending(reason, extra) {
    const cb = rewardedPending;
    rewardedPending = null;
    if (cb) cb(Object.assign({ ok: false, reason }, extra || {}));
}
```

```
function ensureRewardedAd() {
  if (rewardedAd || !api.rewardedAd) return rewardedAd;
  try {
    rewardedAd = tt.createRewardedVideoAd({ adUnitId: PLATFORM.REWARDED_AD_UNIT_ID });
    if (rewardedAd.onClose) {
      rewardedAd.onClose((res) => {
        const cb = rewardedPending;
        rewardedPending = null;
        if (!cb) return;
        // isEnded === true 表示完整观看; false 表示中途退出, 不发奖励
        if (res && res.isEnded === false) cb({ ok: false, reason: 'notEnded' });
        else cb({ ok: true });
      });
    }
    if (rewardedAd.onError) {
      rewardedAd.onError((err) => failRewardedPending('error', { err }));
    }
  } catch (e) {
    rewardedAd = null;
  }
  return rewardedAd;
}

/**
 * 播放激励视频。onResult({ok:true}) 完整观看;
 * {ok:false, reason:'unsupported'|'busy'|'notEnded'|'error'|'showFail'}
 */
function showRewardedAd(onResult) {
  const cb = onResult || function () {};
  const ad = ensureRewardedAd();
  if (!ad || typeof ad.show !== 'function') { cb({ ok: false, reason: 'unsupported' }); return; }
  if (rewardedPending) { cb({ ok: false, reason: 'busy' }); return; }
  rewardedPending = cb;
  try {
    const p = ad.show();
    if (p && typeof p.catch === 'function') {
      p.catch(() => {
        // 首次 show 失败: load 后重试一次
        try {
          const q = (typeof ad.load === 'function') ? ad.load() : null;
          if (q && typeof q.then === 'function') {
            q.then(() => ad.show()).catch((e) => failRewardedPending('showFail', { err: e }));
          } else {
            failRewardedPending('showFail');
          }
        }
      });
    } catch (e) {
      failRewardedPending('showFail', { err: e });
    }
  }
};
```

```
    } catch (e) {
      failRewardedPending('showFail', { err: e });
    }
  }

/* ===== 插屏广告 ===== */

let interstitialAd = null;
let lastInterstitialAt = 0;

/**
 * 展示插屏广告（带最小间隔节流）。适合在游戏结束等自然停顿点调用。
 * @param {boolean} force 忽略间隔限制强制展示
 */
function showInterstitialAd(force) {
  if (!api.interstitialAd) return;
  const now = Date.now();
  // lastInterstitialAt=0 表示从未展示过，首次不节流
  if (!force && lastInterstitialAt !== 0 && now - lastInterstitialAt < PLATFORM.INTERSTITIAL_MIN_INTERVAL) return;
  try {
    if (!interstitialAd) {
      interstitialAd = tt.createInterstitialAd({ adUnitId: PLATFORM.INTERSTITIAL_AD_UNIT_ID });
      if (interstitialAd.onError) interstitialAd.onError(() => { /* 无广告填充等情况，静默 */ });
      if (interstitialAd.onClose) interstitialAd.onClose(() => { /* 关闭即恢复游戏，无需处理 */ });
    }
    lastInterstitialAt = now;
    if (typeof interstitialAd.show === 'function') {
      const p = interstitialAd.show();
      if (p && typeof p.catch === 'function') {
        p.catch(() => {
          try {
            if (typeof interstitialAd.load === 'function') {
              const q = interstitialAd.load();
              if (q && typeof q.then === 'function') q.then(() => interstitialAd.show()).catch(() => {});
            }
          } catch (e) { /* 忽略 */ }
        });
      }
    } else if (typeof interstitialAd.load === 'function') {
      interstitialAd.load(); // 预加载，下次再展示
    }
  } catch (e) { /* 忽略广告异常 */ }
}

/* ===== 订阅消息 ===== */

/** 请求订阅消息（模板 ID 需在开发者后台「功能→订阅消息」中创建并填入 config.js） */
function requestSubscribe(onResult) {
  const cb = onResult || function () {};
  if (!api.subscribe) { cb({ ok: false, reason: 'unsupported' }); return; }
```

```
try {
    tt.requestSubscribeMessage({
        tmplIds: PLATFORM.SUBSCRIBE_TMPL_IDS,
        success: (res) => cb({ ok: true, res }),
        fail: (res) => cb({ ok: false, reason: 'fail', res }),
    });
} catch (e) {
    cb({ ok: false, reason: 'error', err: e });
}
}

/* ===== 添加到桌面 ===== */

/** 引导用户将小游戏添加到手机桌面 */
function addToDesktop(onResult) {
    const cb = onResult || function () {};
    if (!api.addToDesktop) { cb({ ok: false, reason: 'unsupported' }); return; }
    try {
        tt.addToDesktop({
            success: (res) => cb({ ok: true, res }),
            fail: (res) => cb({ ok: false, reason: 'fail', res }),
        });
    } catch (e) {
        cb({ ok: false, reason: 'error', err: e });
    }
}

/* ===== 内购（需版本号，默认隐藏入口） ===== */

/**
 * 购买金币（道具直购）。仅当 PLATFORM.ENABLE_IAP=true 且宿主支持时发起。
 * onResult({ok:true}) 支付成功（发货以服务端回调为准，此处仅做客户端到账）。
 */
function buyCoins(onResult) {
    const cb = onResult || function () {};
    if (!PLATFORM.ENABLE_IAP || !api.iap) { cb({ ok: false, reason: 'unsupported' }); return; }
    const p = PLATFORM.IAP;
    try {
        tt.requestMidasPaymentGameItem({
            buyQuantity: p.coins,      // 购买数量
            zoneId: p.zoneId,          // 游戏分区
            currencyType: 'CNY',
            platform: 'android',
            productId: p.productId,    // 道具 ID
            goodsPrice: p.priceFen,    // 道具价格（分）
            outTradeNo: 'gc' + Date.now() + Math.floor(Math.random() * 10000), // 商户订单号
            attach: 'coins',
            success: (res) => cb({ ok: true, res }),
            fail: (res) => cb({ ok: false, reason: 'fail', res }),
        });
    }
}
```

```
    } catch (e) {
      cb({ ok: false, reason: 'error', err: e });
    }
  }

/* ===== 通用确认弹窗 ===== */

/** tt.showModal 封装; 宿主不支持弹窗时直接按「确认」处理 (降级) */
function confirm(title, content, cb) {
  const done = (ok) => { try { cb(ok); } catch (e) { /* 忽略 */ } };
  try {
    if (api.showModal) {
      tt.showModal({
        title: title,
        content: content,
        confirmText: '确定',
        cancelText: '取消',
        success: (res) => done(!(res && res.confirm)),
        fail: () => done(false),
      });
      return;
    }
  } catch (e) { /* 走降级 */ }
  done(true);
}

module.exports = {
  api,
  init,
  onShow,
  // 钱包
  getCoins, addCoins, spendCoins,
  // 侧边栏复访
  getSidebarState, goSidebar, claimSidebarReward,
  // 广告
  showRewardedAd, showInterstitialAd,
  // 订阅消息 / 桌面 / 内购
  requestSubscribe, addToDesktop, buyCoins,
  // 工具
  confirm,
};

/* ===== 文件: js/rank.js ===== */
/**
 * 排行榜模块
 * - 本机排行: tt.setStorageSync 持久化 TOP10 (始终可用)
 * - 好友排行: tt.setUserCloudStorage 托管分数, 开放数据域内拉取好友数据渲染
 * 所有 tt.* 调用均有守卫, API 不可用时静默降级, 不影响游戏本体。
 */
const TT = (typeof tt !== 'undefined') ? tt : null;
```

```
const KEY_BEST = 'gravcube.best';
const KEY_LOCAL = 'gravcube.localRank';

function safeGet(key, def) {
  try {
    if (TT && TT.getStorageSync) {
      const v = TT.getStorageSync(key);
      return (v === '' || v === undefined || v === null) ? def : v;
    }
  } catch (e) { /* 忽略存储异常 */ }
  return def;
}

function safeSet(key, value) {
  try {
    if (TT && TT.setStorageSync) TT.setStorageSync(key, value);
  } catch (e) { /* 忽略存储异常 */ }
}

/** 本机 TOP10 列表 [{score, level, lines, ts}], 按分数降序 */
function getLocal() {
  const v = safeGet(KEY_LOCAL, []);
  return Array.isArray(v) ? v : [];
}

/** 历史最高分 */
function getBest() {
  const v = safeGet(KEY_BEST, 0);
  return typeof v === 'number' && isFinite(v) ? v : 0;
}

/**
 * 提交一局成绩
 * @returns {{isBest:boolean, best:number, local:Array}}
 */
function submit(score, level, lines) {
  const prevBest = getBest();
  const isBest = score > 0 && score > prevBest;
  if (isBest) safeSet(KEY_BEST, score);

  if (score > 0) {
    const list = getLocal();
    list.push({ score, level, lines, ts: Date.now() });
    list.sort((a, b) => b.score - a.score);
    safeSet(KEY_LOCAL, list.slice(0, 10));
  }

  // 云端托管数据: 供开放数据域的好友排行榜使用
  try {
    if (TT && TT.setUserCloudStorage) {
```

```
TT.setUserCloudStorage({
  KVDataList: [
    { key: 'score', value: String(score) },
    { key: 'level', value: String(level) },
    { key: 'lines', value: String(lines) },
    { key: 'update_time', value: String(Math.floor(Date.now() / 1000)) },
  ],
  fail() {},
});
}
} catch (e) { /* 云端不可用时仅保留本机排行 */ }

return { isBest, best: Math.max(prevBest, score), local: getLocal() };
}

/* ----- 开放数据域（好友排行） ----- */

function getOpenDataContext() {
  try {
    if (TT && TT.getOpenDataContext) return TT.getOpenDataContext();
  } catch (e) { /* 忽略 */ }
  return null;
}

/**
 * 通知开放数据域渲染好友排行榜
 * @param width/height 渲染区域尺寸（物理像素）
 * @returns 共享画布（可能为 null，调用方需判空）
 */
function showFriendRank(width, height) {
  const odc = getOpenDataContext();
  if (!odc) return null;
  try {
    if (odc.postMessage) {
      odc.postMessage({ type: 'renderRank', width: Math.floor(width), height: Math.floor(height) });
    }
  } catch (e) { /* 忽略 */ }
  return odc.canvas || null;
}

/** 离开排行榜页面时通知开放数据域清空共享画布 */
function hideFriendRank() {
  const odc = getOpenDataContext();
  if (!odc) return;
  try {
    if (odc.postMessage) odc.postMessage({ type: 'hide' });
  } catch (e) { /* 忽略 */ }
}

/* ----- 分享 ----- */
```

```
function share(score) {
  try {
    if (TT && TT.shareAppMessage) {
      TT.shareAppMessage({
        title: score > 0
          ? `我在《引力方块》拿了 ${score} 分，四向重力你能撑几关? `
          : '引力方块：方块从中心掉落，重力随机四向，来挑战! ',
        query: 'from=share',
        fail() {},
      });
    }
  } catch (e) { /* 忽略 */ }
}

module.exports = { getLocal, getBest, submit, showFriendRank, hideFriendRank, share };
/* ===== 文件: js/main.js ===== */
/**
 * 主控制器：初始化画布、游戏循环、输入分发、场景切换
 */
const GameCore = require('./gamecore.js');
const Render = require('./render.js');
const Input = require('./input.js');
const rank = require('./rank.js');
const platform = require('./platform.js');
const { DIRS, PERP, PLATFORM } = require('./config.js');

const TT = (typeof tt !== 'undefined') ? tt : null;

class Main {
  constructor() {
    if (!TT || !TT.createCanvas) {
      throw new Error('tt API 不可用：本项目需在抖音开发者工具/真机中运行');
    }
    // 首次调用 createCanvas 返回上屏画布
    this.canvas = TT.createCanvas();
    this.ctx = this.canvas.getContext('2d');

    const info = TT.getSystemInfoSync();
    this.w = info.windowWidth || 375;
    this.h = info.windowHeight || 667;
    this.dpr = Math.min(info.pixelRatio || 2, 3);
    this.canvas.width = Math.round(this.w * this.dpr);
    this.canvas.height = Math.round(this.h * this.dpr);
    this.ctx.setTransform(this.dpr, 0, 0, this.dpr, 0, 0);

    this.L = Render.buildLayout(this.w, this.h);
    this.core = new GameCore();

    this.state = 'menu'; // menu | playing | paused | gameover | rank | help | sidebar
  }
}
```

```
this.best = rank.getBest();
this.localRank = rank.getLocal();
this.friendCanvas = null;
this.newBest = false;
this.fx = []; // 消行闪光 [{cells, t, dur}]
this.toast = null; // 浮字 {text, t, dur}
this.ptoast = null; // 全局平台提示 {text, t, dur} (所有场景可见)
this.last = Date.now();
this._rankReturn = 'menu';
this.coins = platform.getCoins();
this.reviveUsed = false; // 本局是否已用过复活 (每局限一次)

this.input = new Input(TT);
this.input.onTap = (x, y) => this.handleTap(x, y);
this.input.onSwipe = (dx, dy, sx, sy) => this.handleSwipe(dx, dy, sx, sy);

// 从侧边栏等入口回到前台时刷新金币/状态 (侧边栏复访奖励依赖最新 onShow 信息)
platform.onShow(() => { this.coins = platform.getCoins(); });

// 切后台自动暂停
try {
  if (TT.onHide) TT.onHide(() => this.autoPause());
} catch (e) { /* 忽略 */ }
try {
  if (TT.onError) TT.onError((err) => console.error('[game error]', err));
} catch (e) { /* 忽略 */ }

this._raf = (typeof requestAnimationFrame === 'function')
  ? requestAnimationFrame
  : (cb) => setTimeout(() => cb(Date.now()), 16);
this.loop = this.loop.bind(this);
this._raf(this.loop);
}

/* ----- 场景切换 ----- */

autoPause() {
  if (this.state === 'playing') this.state = 'paused';
}

setState(s) {
  if (this.state === 'rank' && s !== 'rank') rank.hideFriendRank();
  this.state = s;
  if (s === 'rank') this.enterRank();
}

enterRank() {
  this.localRank = rank.getLocal();
  this.best = rank.getBest();
  const A = this.L.rankArea;
```

```
this.friendCanvas = rank.showFriendRank(A.w * this.dpr, A.h * this.dpr);
}

startGame() {
  this.core.reset();
  this.fx = [];
  this.toast = null;
  this.newBest = false;
  this.reviveUsed = false;
  this.state = 'playing';
}

/* ----- 平台能力（侧边栏/桌面/订阅/广告/内购，详见 js/platform.js） ----- */

/** 供渲染/命中检测使用的场景状态 */
sceneOpts() {
  const st = platform.getSidebarState();
  return {
    canRevive: this.canRevive(),
    sidebarSupported: st.supported,
    sidebarClaimable: st.claimable,
    sidebarClaimedToday: st.claimedToday,
  };
}

/** 本局是否还能复活（每局限一次；有广告/金币/内购任一途径即可） */
canRevive() {
  if (this.state !== 'gameover' || this.reviveUsed) return false;
  return platform.api.rewardedAd
    || platform.getCoins() >= PLATFORM.REVIVE_COIN_COST
    || (PLATFORM.ENABLE_IAP && platform.api.iap);
}

/** 全局浮动提示 */
notify(msg) {
  this.ptoast = { text: msg, t: 0, dur: 2200 };
}

/** 复活入口：优先看激励视频；广告不可用则金币复活；金币不足且已开通内购则引导购买 */
tryRevive() {
  if (this.state !== 'gameover' || this.reviveUsed) return;
  if (platform.api.rewardedAd) {
    this.notify('正在加载激励视频...');
    platform.showRewardedAd((res) => {
      if (res.ok) { this.doRevive('复活成功，继续加油! '); return; }
      if (res.reason === 'notEnded') { this.notify('完整观看广告才能复活'); return; }
      this.reviveByCoins();
    });
  } else {
    this.reviveByCoins();
  }
}
```

```
    }  
  }  
  
  reviveByCoins() {  
    const cost = PLATFORM.REVIVE_COIN_COST;  
    const coins = platform.getCoins();  
    if (coins >= cost) {  
      platform.confirm('金币复活', '使用 ' + cost + ' 金币复活并继续本局?', (ok) => {  
        if (!ok) return;  
        if (platform.spendCoins(cost)) {  
          this.coins = platform.getCoins();  
          this.doRevive('复活成功, 继续加油! ');  
        } else {  
          this.notify('金币不足');  
        }  
      });  
      return;  
    }  
    if (PLATFORM.ENABLE_IAP && platform.api.iap) {  
      const p = PLATFORM.IAP;  
      platform.confirm('金币不足',  
        '复活需要 ' + cost + ' 金币 (当前 ' + coins + ')。花 ' +  
        (p.priceFen / 100).toFixed(2) + ' 元购买 ' + p.coins + ' 金币?',  
        (ok) => {  
          if (!ok) return;  
          platform.buyCoins((res) => {  
            if (res.ok) {  
              this.coins = platform.addCoins(p.coins);  
              this.notify('购买成功');  
              this.reviveByCoins();  
            } else {  
              this.notify(res.reason === 'unsupported' ? '内购未开通' : '购买未完成');  
            }  
          });  
        });  
      return;  
    }  
    this.notify('复活失败: 广告未就绪且金币不足');  
  }  
  
  doRevive(msg) {  
    if (this.reviveUsed) return;  
    if (this.core.revive()) {  
      this.reviveUsed = true;  
      this.newBest = false;  
      this.fx = [];  
      this.state = 'playing';  
      this.notify(msg);  
      this.vibrate(20);  
    } else {
```

```
        this.notify('复活失败');
    }
}

/** 免费金币：完整观看激励视频得金币 */
earnCoinsByAd() {
    if (!platform.api.rewardedAd) { this.notify('当前环境不支持激励视频'); return; }
    this.notify('正在加载激励视频...');
    platform.showRewardedAd((res) => {
        if (res.ok) {
            this.coins = platform.addCoins(PLATFORM.AD_REWARD_COINS);
            this.notify('金币 +' + PLATFORM.AD_REWARD_COINS);
            this.vibrate(15);
        } else if (res.reason === 'notEnded') {
            this.notify('完整观看广告才能领取');
        } else {
            this.notify('广告未就绪，请稍后再试');
        }
    });
}

/** 侧边栏复访任务面板按钮：可领奖->领奖；否则跳转侧边栏（官方指引的复访动线） */
onSidebarAction() {
    const st = platform.getSidebarState();
    if (st.claimable) {
        if (platform.claimSidebarReward()) {
            this.coins = platform.getCoins();
            this.notify('领取成功：金币 +' + PLATFORM.SIDEBAR_REWARD_COINS);
            this.vibrate(20);
        } else {
            this.notify('今日已领取');
        }
    }
    return;
}
if (!st.supported) { this.notify('当前宿主不支持侧边栏'); return; }
// 跳转抖音首页侧边栏（审核要求：必须使用 tt.navigateToScene）
platform.goSidebar((res) => {
    if (!res.ok) this.notify('跳转侧边栏失败');
});
}

/* ----- 输入 ----- */

handleTap(x, y) {
    const btns = Render.sceneButtons(this.state, this.L, this.sceneOpts());
    for (let i = 0; i < btns.length; i++) {
        const b = btns[i];
        if (x >= b.x && x <= b.x + b.w && y >= b.y && y <= b.y + b.h) {
            this.onButton(b.id);
            return;
        }
    }
}
```

```
    }
  }
  if (this.state === 'help' || this.state === 'sidebar') { this.setState('menu'); return; }
  if (this.state === 'playing') {
    const B = this.L.board;
    if (x >= B.x && x <= B.x + B.size && y >= B.y && y <= B.y + B.size) {
      if (this.core.rotate()) this.vibrate(8);
    }
  }
}

handleSwipe(dx, dy, sx, sy) {
  if (this.state !== 'playing' || !this.core.canControl()) return;
  const B = this.L.board;
  // 仅响应从棋盘区域起始的滑动
  if (!(sx >= B.x && sx <= B.x + B.size && sy >= B.y && sy <= B.y + B.size)) return;

  const dir = this.core.current.dir;
  const d = DIRS[dir];
  const p = PERP[dir];
  const dotG = dx * d.x + dy * d.y; // 沿重力分量
  const dotP = dx * p.x + dy * p.y; // 垂直分量

  if (Math.abs(dotG) >= Math.abs(dotP)) {
    if (dotG > 30) { this.core.hardDrop(); this.vibrate(15); }
  } else {
    // 按格子实际尺寸换算: 拖动约一个格宽 = 移动一格 (适配任意棋盘尺寸)
    const steps = Math.min(8, Math.max(1, Math.round(Math.abs(dotP) / B.cell)));
    const sign = dotP > 0 ? 1 : -1;
    let moved = false;
    for (let i = 0; i < steps; i++) { if (this.core.movePerp(sign)) moved = true; else break; }
    if (moved) this.vibrate(5);
  }
}

onButton(id) {
  switch (id) {
    case 'start':
    case 'retry':
    case 'restart':
      this.startGame();
      break;
    case 'resume':
      this.state = 'playing';
      break;
    case 'pause':
      if (this.state === 'playing') this.state = 'paused';
      break;
    case 'tomenu':
      this.setState('menu');
  }
}
```

```
        break;
    case 'rank':
        this._rankReturn = (this.state === 'gameover') ? 'gameover' : 'menu';
        this.setState('rank');
        break;
    case 'back':
        this.setState(this._rankReturn || 'menu');
        break;
    case 'help':
        this.setState('help');
        break;
    case 'share':
        rank.share(this.core.score);
        break;
    /* ---- 平台能力按钮 ---- */
    case 'sidebarGift':
        this.setState('sidebar');
        break;
    case 'sidebarAction':
        this.onSidebarAction();
        break;
    case 'sidebarClose':
        this.setState('menu');
        break;
    case 'desktop':
        platform.addToDesktop((res) => {
            if (res.ok) this.notify('已发起添加到桌面');
            else this.notify(res.reason === 'unsupported' ? '当前环境不支持添加到桌面' : '添加到桌面未完成');
        });
        break;
    case 'subscribe':
        platform.requestSubscribe((res) => {
            if (res.ok) this.notify('已订阅消息提醒');
            else this.notify(res.reason === 'unsupported' ? '当前环境不支持订阅消息' : '订阅未完成');
        });
        break;
    case 'freeCoins':
        this.earnCoinsByAd();
        break;
    case 'revive':
        this.tryRevive();
        break;
    case 'left':
        if (this.core.movePerp(-1)) this.vibrate(5);
        break;
    case 'right':
        if (this.core.movePerp(1)) this.vibrate(5);
        break;
    case 'rotate':
        if (this.core.rotate()) this.vibrate(8);
```

```
        break;
    case 'drop':
        if (this.core.canControl()) { this.core.hardDrop(); this.vibrate(15); }
        break;
    default:
        break;
    }
}

vibrate(ms) {
    try {
        if (ms >= 50 && TT.vibrateLong) TT.vibrateLong({ fail() {} });
        else if (TT.vibrateShort) TT.vibrateShort({ fail() {} });
    } catch (e) { /* 忽略 */ }
}

/* ----- 事件与特效 ----- */

processEvents() {
    const evs = this.core.drainEvents();
    for (let i = 0; i < evs.length; i++) {
        const ev = evs[i];
        if (ev.type === 'clear') {
            this.fx.push({ cells: ev.cells, t: 0, dur: 300 });
            this.toast = { text: ev.count >= 2 ? ('消除 x' + ev.count + '!' ) : '消除! ', t: 0, dur: 900 };
            this.vibrate(20);
        } else if (ev.type === 'levelup') {
            this.toast = { text: '第 ' + (ev.level - 1) + ' 关合格! 第 ' + ev.level + ' 关提速', t: 0, dur: 1600 };
            this.vibrate(30);
        } else if (ev.type === 'gameover') {
            const r = rank.submit(this.core.score, this.core.level, this.core.lines);
            this.best = r.best;
            this.localRank = r.local;
            this.newBest = r.isBest;
            this.reviveUsed = false;
            this.vibrate(80);
            this.setState('gameover');
            // 插屏广告: 游戏结束是自然停顿点 (platform 内部有最小间隔节流)
            platform.showInterstitialAd();
        }
    }
}

updateFx(dt) {
    for (let i = this.fx.length - 1; i >= 0; i--) {
        this.fx[i].t += dt;
        if (this.fx[i].t >= this.fx[i].dur) this.fx.splice(i, 1);
    }
    if (this.toast) {
        this.toast.t += dt;
    }
}
```

```
        if (this.toast.t >= this.toast.dur) this.toast = null;
    }
    if (this.ptoast) {
        this.ptoast.t += dt;
        if (this.ptoast.t >= this.ptoast.dur) this.ptoast = null;
    }
}

/* ----- 主循环 ----- */

loop() {
    const now = Date.now();
    const dt = Math.min(100, now - this.last);
    this.last = now;

    if (this.state === 'playing') {
        this.core.update(dt);
        this.processEvents();
    }
    this.updateFx(dt);
    Render.draw(this);
    this._raf(this.loop);
}
}

module.exports = Main;
/* ===== 文件: openDataContext/index.js ===== */
/**
 * 开放数据域 (好友排行榜)
 * 运行在独立的 JS 上下文中, 只能使用 tt 的开放数据相关 API:
 *   tt.getSharedCanvas / tt.onMessage / tt.getFriendCloudStorage
 * 主域通过 postMessage({type:'renderRank', width, height}) 请求渲染。
 */
const sharedCanvas = tt.getSharedCanvas();
const ctx = sharedCanvas.getContext('2d');

let areaW = 300;
let areaH = 400;
let friends = [];
let renderToken = 0; // 防止异步头像回调渲染过期数据

tt.onMessage((msg) => {
    if (!msg) return;
    if (msg.type === 'renderRank') {
        areaW = Math.max(60, msg.width || sharedCanvas.width || 300);
        areaH = Math.max(60, msg.height || sharedCanvas.height || 400);
        sharedCanvas.width = areaW;
        sharedCanvas.height = areaH;
        fetchFriends();
    } else if (msg.type === 'hide') {
```

```
renderToken++;
ctx.clearRect(0, 0, sharedCanvas.width, sharedCanvas.height);
}
});

function fetchFriends() {
  const token = ++renderToken;
  drawLoading(token);
  try {
    if (typeof tt.getFriendCloudStorage !== 'function') {
      drawEmpty(token, '当前基础库不支持好友数据');
      return;
    }
    tt.getFriendCloudStorage({
      keyList: ['score', 'level', 'lines'],
      success(res) {
        if (token !== renderToken) return;
        friends = normalize(res && res.data ? res.data : []);
        renderList(token);
        loadAvatars(token);
      },
      fail() {
        if (token !== renderToken) return;
        drawEmpty(token, '暂无好友数据, 邀请好友一起玩吧');
      },
    });
  } catch (e) {
    if (token === renderToken) drawEmpty(token, '好友数据不可用');
  }
}

function normalize(list) {
  const rows = [];
  for (let i = 0; i < list.length; i++) {
    const it = list[i];
    let score = 0, level = 0, lines = 0;
    const kvs = it.KVDataList || [];
    for (let j = 0; j < kvs.length; j++) {
      const kv = kvs[j];
      if (kv.key === 'score') score = parseInt(kv.value, 10) || 0;
      else if (kv.key === 'level') level = parseInt(kv.value, 10) || 0;
      else if (kv.key === 'lines') lines = parseInt(kv.value, 10) || 0;
    }
    rows.push({
      nickname: it.nickname || '神秘玩家',
      avatarUrl: it.avatarUrl || '',
      score: score,
      level: level,
      lines: lines,
      img: null,
    });
  }
}
```

```
    });
  }
  rows.sort((a, b) => b.score - a.score);
  return rows.slice(0, 30);
}

function loadAvatars(token) {
  if (typeof tt.createImage !== 'function') return;
  for (let i = 0; i < friends.length; i++) {
    const f = friends[i];
    if (!f.avatarUrl || !f.img) continue;
    try {
      const img = tt.createImage();
      img.onload = () => {
        f.img = img;
        if (token === renderToken) renderList(token);
      };
      img.onerror = () => { /* 头像加载失败则显示首字 */ };
      img.src = f.avatarUrl;
    } catch (e) { /* 忽略 */ }
  }
}

/* ----- 绘制 ----- */

function drawLoading(token) {
  ctx.clearRect(0, 0, areaW, areaH);
  centerText('好友排行加载中...', areaH / 2, 14, 'rgba(255,255,255,0.6)', false);
}

function drawEmpty(token, msg) {
  ctx.clearRect(0, 0, areaW, areaH);
  centerText('好友排行', 30, 15, '#ffd54f', true);
  centerText(msg, areaH / 2, 13, 'rgba(255,255,255,0.55)', false);
}

function centerText(str, y, size, color, bold) {
  ctx.save();
  ctx.font = (bold ? 'bold ' : '') + size + 'px sans-serif';
  ctx.fillStyle = color;
  ctx.textAlign = 'center';
  ctx.textBaseline = 'middle';
  ctx.fillText(str, areaW / 2, y);
  ctx.restore();
}

function renderList(token) {
  ctx.clearRect(0, 0, areaW, areaH);
  if (!friends.length) { drawEmpty(token, '暂无好友数据, 邀请好友一起玩吧'); return; }
```

```
ctx.save();
ctx.font = 'bold 15px sans-serif';
ctx.fillStyle = '#ffd54f';
ctx.textAlign = 'left';
ctx.textBaseline = 'middle';
ctx.fillText('好友排行', 12, 22);
ctx.restore();

const rowH = Math.min(44, Math.max(28, (areaH - 44) / Math.min(friends.length, 8)));
const n = Math.min(friends.length, Math.floor((areaH - 40) / rowH));
for (let i = 0; i < n; i++) {
  const f = friends[i];
  const y = 44 + i * rowH + rowH / 2;
  const cx = 14;

  // 名次
  ctx.save();
  ctx.font = 'bold 14px sans-serif';
  ctx.textAlign = 'left';
  ctx.textBaseline = 'middle';
  ctx.fillStyle = i === 0 ? '#ffd54f' : (i === 1 ? '#cfd8dc' : (i === 2 ? '#ffab40' : 'rgba(255,255,255,0.5)'));
  ctx.fillText(String(i + 1), cx, y);
  ctx.restore();

  // 头像
  const ax = cx + 26, ar = rowH * 0.34;
  if (f.img) {
    try {
      ctx.save();
      ctx.beginPath();
      ctx.arc(ax + ar, y, ar, 0, Math.PI * 2);
      ctx.clip();
      ctx.drawImage(f.img, ax, y - ar, ar * 2, ar * 2);
      ctx.restore();
    } catch (e) { drawAvatarFallback(f, ax + ar, y, ar); }
  } else {
    drawAvatarFallback(f, ax + ar, y, ar);
  }

  // 昵称
  ctx.save();
  ctx.font = '13px sans-serif';
  ctx.fillStyle = 'rgba(255,255,255,0.9)';
  ctx.textAlign = 'left';
  ctx.textBaseline = 'middle';
  const nameX = ax + ar * 2 + 10;
  let nick = f.nickname;
  if (nick.length > 7) nick = nick.slice(0, 7) + '...';
  ctx.fillText(nick, nameX, y - 7);
  ctx.font = '10px sans-serif';
```

```
    ctx.fillStyle = 'rgba(255,255,255,0.45)';
    ctx.fillText('第' + f.level + '关 · ' + f.lines + '行', nameX, y + 9);
    ctx.restore();

    // 分数
    ctx.save();
    ctx.font = 'bold 15px sans-serif';
    ctx.fillStyle = 'ffffff';
    ctx.textAlign = 'right';
    ctx.textBaseline = 'middle';
    ctx.fillText(String(f.score), areaW - 14, y);
    ctx.restore();
}
}

function drawAvatarFallback(f, cx, cy, r) {
    ctx.save();
    ctx.beginPath();
    ctx.arc(cx, cy, r, 0, Math.PI * 2);
    ctx.fillStyle = 'rgba(255,255,255,0.15)';
    ctx.fill();
    ctx.font = 'bold ' + r + 'px sans-serif';
    ctx.fillStyle = 'rgba(255,255,255,0.8)';
    ctx.textAlign = 'center';
    ctx.textBaseline = 'middle';
    ctx.fillText((f.nickname || '?').slice(0, 1), cx, cy + 1);
    ctx.restore();
}
```